Instituto Tecnológico y de Estudios Superiores de Monterrey Campus Guadalajara

Desarrollo de aplicaciones avanzadas de ciencias computacionales (Gpo 501)

Analizador Léxico

Diego Enrique Vargas Ramírez - A01635782

Sayid Isaías Valdivia Frausto - A01643383

Rafael Romo Muñoz - A01643137

Santos

Bryan Ithan Landín Lara - A01636271

Tabla de contenidos

Tabla de contenidos **2**
1. Introducción **3**
1.1.Resumen 3
1.1.Notación 3
2.Análisis **6**
2.1 Entrada de Archivo 8
2.2 Definición del lexema 8
2.3 Token Classification 8
2.4 Error Outputting 8
2.? Token and table output 8
3. Design **9**
4. Implementation **14**
5. Verification and Validation **14**
6. References **15**

Rubrica de codigo

Aspect	100	80	60	40	20	0	Weight
Software Complies with Requirements	1. Software runs. 2. Scanner recognizes all tokens. 3. Scanner recognizes invalid symbols and provides the corresponding error message. 4. Lexical analyzer provides a list of tokens of the source file scanned. 5. Scanner provides the Symbol Table, for all valid Tokens.	Software runs and recognizes all tokens but does not comply with one of the other features.	Software runs and recognizes all tokens but does not comply with two or more of the other features.	Does not apply	Does not apply	Software **does not run**, or failed to identify all tokens, or it was developed with **API for RE**, or the implementation **does not** use **lex**.	80
Comments in Source Code	1. Lexical analyzer source code is extraordinarily explained and documented 2. All source code files include a description of its functionality and relation with other source code files. 3. All functions/methods and/or classes are clearly and completely described. 4. Comments are unambiguous, complete, and correct with respect to analysis and design.	All source code is commented, however, is hard to understand the meaning and support that the comments provide to each section.	Comments are incomplete, ambiguous, or incorrect.	Does not apply	Extremely poor comments description.	Source code does not include comments.	5
Traceability	1. Every single functional requirement can be mapped directly to a specific piece of code. 2. The code complies with the design. 3. The design can be mapped directly to the implementation	Does not apply	Does not apply	Does not apply	Does not apply	**Poor traceability**, or developed with **API for RE**, or the implementation **does not** use **lex**.	5
Testing	The scanner software passes all test cases given by the professor.	Does not apply	Does not apply	Does not apply	Does not apply	The scanner software does not pass all test cases given by the professor.	10

Rubrica documento

Aspect	100	80	60	40	20	0	Weight
Analysis	1. Complete informal specification of the functional requirements for the scanner, i.e., Informal description of the lexical components. 2. Formal Specification using Regular Expression for the lexical components. 3. Description and justification of tokens and their identification. 4. Complete description and justification for every error type messages generated by scanner. 5. This section contains a comprehensive, coherent and unambiguous narrative that relates all its contents.	1. There are minor discrepancies with the specification of the requirements. However, the information provided is sound and it is traceable for the complete development process. 2. The phase has a poor narrative.	1. There are mayor discrepancies with the specification of the requirements. 2. It is very difficult to trace this specification for the complete development process. 3. However the scanner works.	NA	NA	Specification is ambiguous, or inconsistent, or incomplete, or incorrect, or Software analysis was developed with API for RE, or the implementation does not use lex.	20
Design	1. Informal description. 2. The automata of the language. 3. The most efficient Transition Table. 4. List of Token IDs 5. Symbol Table management, its algorithm and data structures. 6. Traceable to the analysis model. 7. This section contains a comprehensive, coherent and unambiguous narrative that relates all its contents.	1. There are minor discrepancies with the specification of the requirements. However, the information provided is sound and it is traceable for the complete development process. 2. The phase has a poor narrative.	Traceability to the analysis model is not clear. However the scanner works.	NA	NA	Design is ambiguous, or inconsistent, or incomplete, or incorrect, or Software design was developed with API for RE, or the implementation does not use lex.	20
Implementation with lex.	1. Lex code is completely and correctly traceable to the analysis & design provided. 2. Every lex section is completely and correctly explained. 3. This section contains a comprehensive, coherent and unambiguous narrative that relates all its contents.	1. There are minor discrepancies with the specification of the requirements. However, the information provided is sound and it is traceable for the complete development process. 2. The phase has a poor narrative.	Traceability to the analysis model is not clear. However the scanner works.	NA	NA	Scanner does not work and the lex sections are not well developed and or explained; or the implementation does not use lex	40
Testing	1. Informal description. 2. Software Test Cases design. 3. Justification.	NA	NA.	NA	NA	The section does not exist.	3

1. Introducción

1.1.- Resumen

En este documento describiremos la funcionalidad y la implementación de un analizador léxico para un compilador del lenguaje de programación Triton. Concretamente, el compilador se utilizará para verificar la validez del código generado por modelos de aprendizaje de lenguaje (LLM). El documento analizará los requisitos del analizador léxico, proporcionando especificaciones sobre su funcionalidad, cómo se implementará dicha funcionalidad, así como los diagramas necesarios para ilustrar el sistema.

El objetivo es ofrecer una guía completa sobre el diseño del analizador, así como proporcionar trazabilidad de su diseño tanto para los desarrolladores actuales como para los lectores que deseen trabajar con el diseño o comprender cómo funciona.

1.2.- Notación

Antes de entrar en la especificación formal de nuestro analizador léxico, es importante comprender algunos conceptos fundamentales para el diseño del componente. Empezaremos por las máquinas de estados.

Las máquinas de estados son una forma matemática de representar el comportamiento de un sistema. Como su nombre lo indica, se componen de estados que representan las diferentes condiciones en las que puede encontrarse un sistema. Para movernos entre ellos, necesitamos entradas que determinen las transiciones entre estados. A medida que avanzamos por la máquina, esperamos llegar a un estado final.

Una expresión regular puede entenderse como un patrón de caracteres que se utiliza para buscar coincidencias con dicho patrón dentro de un texto, o cadena en el caso específico del software. Para nuestro analizador léxico, las expresiones regulares son los patrones que se definirán con el fin de determinar el tipo de tokens presentes en el código generado, así como para rechazar cualquier carácter o cadena que no se ajuste a nuestro conjunto definido de expresiones.

Una tabla de transiciones es una representación tabular de la función de transición de un autómata finito, donde las filas representan los estados actuales, las columnas representan los símbolos del alfabeto de entrada, y cada celda indica el estado siguiente. Esta representación tabular es precisamente la que permite traducir el diseño teórico de un autómata a una implementación lógica eficiente.

El modelo utilizado para el desarrollo del analizador léxico se basa en máquinas de estados finitos (FSM), concretamente en autómatas finitos determinísticos (DFA). Este modelo resulta adecuado porque el análisis léxico consiste en reconocer patrones en una secuencia de caracteres, lo cual puede representarse de manera eficiente mediante transiciones de estado.

Se utilizan expresiones regulares para definir formalmente los patrones correspondientes a cada token. A continuación, estas expresiones se transforman en DFA, ya

sea conceptualmente durante la fase de diseño o automáticamente mediante la herramienta lex durante la implementación.

Lex es una herramienta fundamental para el diseño de compiladores, ya que automatiza la creación de analizadores léxicos mediante la conversión de expresiones regulares en autómatas finitos deterministas (AFD); su uso facilita la división clara entre las fases de análisis léxico y sintáctico, simplificando el diseño del compilador, mejorando su eficiencia al generar código optimizado en C, y aumentando la portabilidad al aislar las dependencias del sistema en el analizador léxico, integrándose además de forma efectiva con herramientas como Yacc para el análisis sintáctico.

2. Análisis

2.1.- Descripción informal del lenguaje

El lenguaje Triton es un lenguaje de dominio específico (DSL) similar a Python que incluye estructuras de control, operaciones aritméticas, identificadores y literales, como números y cadenas de caracteres. El analizador léxico debe reconocer estos elementos para validar la estructura del código fuente.

2.2.- Descripción informal del lenguaje

Lexema	**REGEX**	**Justificación**
BLANK	(\s | \t\ | n)	Todo input en blanco es omitido dentro del analizador léxico, ya que el punto es tener una lista de tokens que el analizador sintáctico use.
DEF	def	Reconoce la palabra reservada def, utilizada para declarar funciones en el lenguaje. Indica el inicio de la definición de una función, seguida por su nombre y parámetros.

IF	if	Palabra reservada utilizada para estructuras condicionales.
ELSE	else	Palabra reservada utilizada para estructuras condicionales.
FOR	for	Palabra reservada para ciclos.
WHILE	while	Palabra reservada para ciclos.
IMPORT	import	Permite incluir módulos, librerías o recursos externos dentro del programa.
AS	as	Se utiliza para asignar un alias a un módulo, recurso o valor importado.
IN	in	Indica pertenencia o recorrido dentro de una colección o secuencia.
TRUE	true	Representa el valor booleano verdadero.
FALSE	false	Representa el valor booleano falso.
NONE	none	Representa la ausencia de valor o un valor nulo.
COMMENT	\#[^\n]*	Permite al compilador ignorar anotaciones humanas desde el símbolo # hasta el final de la línea, facilitando la documentación sin afectar el flujo del programa.
DOT	.	Facilita el acceso a espacios de nombres y atributo, fundamental para la estructura modular y el uso de la librería estándar de Triton.
AT SIGN	@	Identifica decoradores (como @triton.jit), que son instrucciones de compilación que

		indican qué funciones deben transformarse en kernels de GPU.
PLUS SIGN	+	Define operador de suma utilizado en expresiones aritméticas.
MINUS SIGN	-	Define el símbolo -, usado para resta o como signo negativo.
MULTIPLICATION SIGN	*	Define el símbolo * utilizado para multiplicación.
DIVISION SIGN	/	Define el símbolo / utilizado para división.
MODULO SIGN	%	Define el símbolo % usado para obtener el residuo de una operación.
GREATER THAN	>	Reconoce el operador > usado en comparaciones.
LESS THAN	<	Reconoce el operador < usado en comparaciones.
EQUAL	=	Operador de asignación que iguala valores.
BITWISE AND	&	Operador usado para realizar una operación AND a nivel de bits.
BITWISE XOR		Operador usado para realizar una operación XOR a nivel de bits.
COLON	:	Reconoce el símbolo :, comúnmente usado para iniciar bloques de código.
COMMA	,	Reconoce la coma ,, usada para separar elementos o parámetros.

OPEN PARENTHESIS	(	Reconoce el paréntesis de apertura (.
CLOSE PARENTHESIS	)	Reconoce el paréntesis de cierre).
OPEN SQUARE BRACKET	[	Reconoce el corchete de apertura [.
CLOSE SQUARE BRACKET	]	Reconoce el corchete de cierre].
NUMBER	-?[0-9][0-9]*	Esta expresión regular permite formatos válidos abreviados en lenguaje de Triton (1., .2) y asegura que no se de el error de un punto solitario, garantizando que se cargue correctamente
IDENTIFIER	([A-Za-z]|_)([A-Za-z]|_|[0-9])*	Esta expresión regular garantiza que los identificadores comienzan con una letra o un guión bajo, seguidos de cualquier combinación de letras, dígitos o guiones bajos, lo cual se ajusta a las convenciones estándar de los lenguajes de programación.
STRING	\"([^\\\n\"]|\\.)*\"	Este expresion regular identifica strings que inicien con un comillas, cualquier carácter excepto barras invertidas

¿Qué debemos lograr aquí?

1. Describir QUE hace el sistema, no el COMO.
    a. Básicamente irnos al detalle que es un analizador léxico
2. Qué inputs y outputs tenemos?

   a. Input: Un Kernel generado por inteligencia artificial
   b. Output: Tablas de símbolos + lista de tokens.
3. ¿Qué funciones tiene que hacer el sistema?
   a. Defines and classifies valid tokens in the form of regular expressions
   b. Accepts and input file to analyze.
   c. Classifies and outputs the input tokens in three tables
      i. Number table
      ii. String table
      iii. Identifier table
   d. As well as outputting a general token list that also includes operators and keywords.
   e. Create and output JSON file with the tables and token list.
4. Comportamientos del sistema.
5. Describir que requiere el cliente
   a. Un scanner con el cual podemos validar el código de Triton generado por LLMs de manera léxica, sintáctica y semántica.
6. Establecer la base para la creación del sueño del sistema de software.
   a. Se hace con el resto de puntos.
7. Establecer requerimientos que se pueden validar (testear) una vez que el software esté construido. (Revisar 8 puntos de las entregas del scanner)

1. **The approved informal description of the lexemes required to achieve the goal.**
2. **The regular expression for each kind of token.**
3. **The automata that recognize the required lexemes.**
4. **Tokens and their identification (Token ID).**
5. **Transition Table.**
6. **Symbol Tables for the required tokens.**
7. **Implementation of the Scanner using UNIX-lex.**
8. **Example of the Scanner Outputs.**

   a. El analizador léxico puede aceptar un archivo de texto para escanear.
   b. El analizador léxico puede arrojar errores respecto a la estructura léxica.
   c. El analizador léxico debe generar un archivo JSON el cual contenga las tablas de símbolos así como la lista de tokens.

8. Debe incluir todos los pasos requeridos para generar un set completo y formal de especificaciones para ellos.
9. Debe incluir una explicación de cada paso del proceso, dejando en claro que se debe hacer y por qué.

¿Qué se tiene que escribir / hacer literalmente?

- ☐ Breve descripción y explicación de los componentes léxico requeridos para cumplir el objetivo.
- ☐ Descripción informal léxica del lenguaje.
- ☐ Especificación formal en la forma de expresiones regulares de cada token.
- ☐ Describir las consideraciones tomadas para estos modelos.
- ☐ Explicar cada regex.

As the document has previously stated, the purpose is to create a lexical analyzer that will work in tandem with other components in order to create a compiler for the Triton programming language. The scanner in particular validates that each of the generated tokens is valid to process in Triton, otherwise just discarding the code and yield an appropriate error message. In this section of the document we will break down the requirements needed for this part of the system to work.

3. Design

¿Qué debemos lograr aquí?

- Convertir los requerimientos a producto.
    - Debe implementar todos los requerimientos explícitos en el análisis.
    - Debe ser legible y entendible para desarrolladores y testers.

    - Dar una imagen completa del sistema especificando dominios de:
        - Datos
        - Funcional
        - De comportamiento.
- El diseño debe exhibir una arquitectura que muestre modularidad.
    - Osea que este dividido en subsistemas
    - Qué requerimientos abarca cada subsistema.
    - Y qué funcionalidades del sistema están definidas por hardware y cuales por software.
- El diseño debe estar conformado por un set de diagramas (diagramas de estado, diagramas de flujo, diagramas de módulo.)
- Pseudocódigo debe ser usado para complementar el diagrama de estado y flujo.
- El diseño debe ser trazable hacia el analisis
    - Por cada requerimiento funcional especificado en la seccion 2, el diseño debe explicar “como” este requerimiento es implementado.
- La implementación debe estar basada 100% en el diseño y ser trazable a este.

¿Qué se tiene que escribir / hacer literalmente?

- ☐ Deterministic Finite Automata que reconoce todos los tokens enumerados en el análisis.
- ☐ Tabla de transiciones más eficiente.
- ☐ IDs de los tokens.
- ☐ Descripción algorítmica y la estructuras de datos requeridas para implementar los componentes del escáner.
    - Como ejemplo cómo se implementa la tabla de símbolos.
- ☐ Diagrama de estado? (Consultar con profe)
- ☐ Diagrama de flujo?
- ☐ Diagrama de módulo?

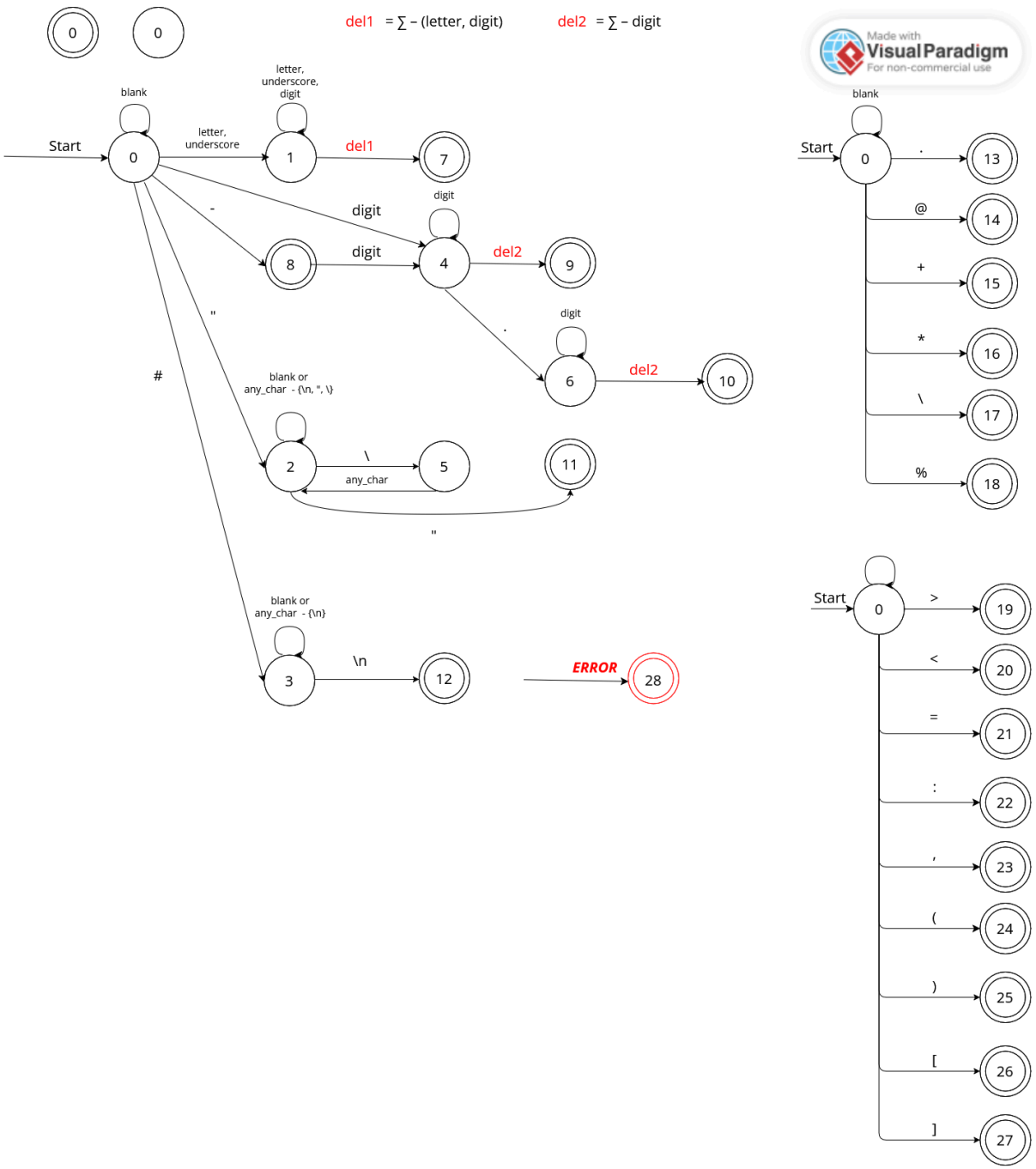

Figure X: Deterministic Finite Automata

Figure X showcases the expected behavior of our token detection. On the right we mostly feature basic operators and like those for mathematical operations or enclosing ones like the opening and closing parenthesis. On the left we feature the logic to classify various types of multicharacter tokens. Up on the top there's the identifier which allows any character or digit as long as we begin with an underscore or character. Next is a minus sign that can be just the operator or used in front of one or more numbers, we can finish this case here and

detect the token as an integer or continue with a dot and more numbers that form a float. Then there's our flow for a valid string surrounded by quotations. The string part of the state machine also features special logic to avoid single backslashes, the syntactic analyzer later handling the validity of the slashes. Finally at the bottom we have the logic for comments, since the base Python programming language doesn't feature multiline comments the process is as simple as detecting the initial hashtag and accepting any char except for a line break.

This is the basis from which our different tokens will be scanned and classified, the lex language converting our regular expression into the necessary state machines to identify the tokens. To further clarify the flow of logic of our

	letter	"	#	digit	-	.	@	+	*	\	%	>	<	=	:	,	(	)	[	]	Blank	Rare Char
0	1	2	3	4	8	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	0	28
1	1	1	7	7	7	7	7	7	7	7	7	7	7	7	7	7	7	7	7	7	7	28
2	2	11	2	2	2	2	2	2	2	5	2	2	2	2	2	2	2	2	2	2	2	28
3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	
4	28	9	9	4	9	9	9	9	9	9	9	9	9	9	9	9	9	9	9	9	9	28
5	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	28
6	28	10	10	6	10	10	10	10	10	10	10	10	10	10	10	10	10	10	10	10	10	28
7																						
8	28	28	28	4	28	28	28	28	28	28	28	28	28	28	28	28	28	28	28	28	28	28
9																						
10																						
11																						
12																						

13																						
14																						
15																						
16																						
17																						
18																						
19																						
20																						
21																						
22																						
23																						
24																						
25																						
26																						
27																						
28																						

To provide a consistent functionality of the scanner, it is also important to assign clear and specific IDs to each of the tokens the system would expect to encounter. The IDs are important both for the expected output of the scanner as well as to keep consistency with said IDs during the creation of the next components of the compiler. Thus, the token IDs would go as follows:

1.- BLANK	18.- MULTIPLICATION SIGN
2.- DEF	19.- DIVISION SIGN
3.- IF	20.- MODULO SIGN
4.- ELSE	21.- GREATER THAN
5.- FOR	22.- LESS THAN
6.- WHILE	23.- EQUAL
7.- IMPORT	24.- BITWISE AND

8.- AS	25.- BITWISE XOR
9.- IN	26.- COLON
10.- TRUE	27.- COMMA
11.- FALSE	28.- OPEN PARENTHESIS
12.- NONE	29.- CLOSE PARENTHESIS
13.- COMMENT	30.- OPEN SQUARE BRACKET
14.- DOT	31.- CLOSE SQUARE BRACKET
15.- AT SIGN	32.- NUMBER
16.- PLUS SIGN	33.- IDENTIFIER
17.- MINUS SIGN	34.- STRING

4. Implementation

¿Qué debemos añadir aquí?

- Código del scanner con cada elemento de código explicado, específicamente las tres secciones de un lex file.
    - Sección de definición
    - Sección de reglas
    - Seccion de codigo

5. Verification and Validation

The student must present a Test model. The test model consists of the set of test cases that are developed during the test case design.

During and after the implementation process, the system being developed must be checked to ensure that it meets its specification and delivers the functionality expected by the customer.

Verification and Validation (V&V) is the name given to these checking processes. V&V starts with requirements reviews and continues through design reviews and hardware and code inspections up to product testing.

Verification and validation is not the same thing, verification deals with “are we building the product right?” while validation deals with “are we building the right product?” Verification involves checking that the system conforms to its specification. The developer must check that it meets its specified functional and non-functional requirements. However, validation aims to ensure that the system meets the expectation of the customer. The ultimate goal of the V&V process is to establish confidence that the system is good enough for its intended use.

In order to perform the V&V, the developer must implement a Test Case Design phase. The test cases are part of system and component testing where the developer designs the test cases (inputs and predicted outputs) that test the system. The goal of this phase is to create a set of test cases that are effective in discovering hardware and software defects and showing that the system meets its requirements. To design a test case, the developer must select a particular feature of the system or component that is to be tested. Then, the developer must select a set of inputs that execute that feature, document the corresponding outputs, and check that the actual and expected outputs are the same.

Provide your own set of test files and their expected results. Your implementation MUST pass your test files as well as the professor’s test cases. Be sure to include snapshots of the Scanner’s output for your test cases, together with the corresponding explanation.

6. References

Any information that is used to develop this document must be listed on a standard bibliography format.

With respect to bibliography, the IEEE Reference Style must be used. This style incorporates common practices of bibliographic references of the scientific and technical fields. This style uses numeric references enclosed on square brackets inserted into the text, whenever the writer needs to link the text to a bibliography entry. The bibliography list must include all the references used on the text. The general structure of an input on the bibliography list is the following:

- Author or authors, begins with the first name followed by the last.

o Title: Every main word starts with a capital letter and all are italic. If the source is not a book or an article, a description of
the source must be included.

o Publisher information: editor and year.

o Page numbers:

In case of articles from scientific journals, the name of the author is followed by the title of the article. The title of the article must be enclosed between quotation marks. Following, the complete name of the journal must be written in italics. Immediately, the volume number as well as the issue number must be included. Finally, the date enclosed in parenthesis, and followed by colon and the pages numbers.

[1] Sashmin Adhikari, "Finite State Machines and Automata Theory," Medium, Apr. 20, 2025. [Online]. Available: https://medium.com/@sashminadhikari/finite-state-machines-and-automata-theory-5fbb51bfe59f. [Accessed: Apr. 27, 2026].

[2] Nagarjoon B, "What Are Regular Expressions, and Why Should You Use Them?," Medium, Aug. 14, 2022. [Online]. Available: https://medium.com/@nagarjoon.b/what-are-regular-expressions-and-why-should-you-use-them-26140fe52bbe. [Accessed: Apr. 27, 2026].